Python
Funções

170

Abstração

- É uma técnica de programação que nos permite pensar em um problema em diversos níveis.
- A ideia é que quando estamos pensando em um problema macroscopicamente, não estamos preocupados com minúcias.
- Dividir para conquistar:
  - Um problema é dividido em diversos subproblemas (menores).
  - Cada subproblema é resolvido individualmente.
  - As soluções dos subproblemas são combinadas em uma solução do problema maior.

171

Programação Estruturada

- É uma disciplina de programação que incorpora o princípio de “Dividir para Conquistar”
  - Programação Orientada a Objetos é outra...
- Programas podem ser divididos em subprogramas
  - Cada subprograma é invocado por meio de um identificador e uma lista de entradas.
    - Especifica como um problema pode ser resolvido em geral.
    - O mesmo subprograma pode ser chamado para resolver diversas instâncias do problema (com valores diferentes).
  - Os resultados computados por um subprograma podem ser combinados com os de outros subprogramas.

172

Definindo funções

- Em Python, subprogramas têm o nome de **funções**. Formato geral:

```
def <nome> ([arg1, arg2, . . .]) :
    <comando_1>
    <comando_2>
    . . .
    <comando_n>
```

- Onde:
  - **nome** é o nome da função.
  - **args** é a lista (de 0, 1 ou mais) valores passados a função para processamento.
  - **comandos** são as instruções a serem executadas pela função.

173

Exemplo

```
def funcao1():
  print("Olá, tudo bem?")

def funcao2():
  print(5 + 2)

funcao1()
funcao2()
```

Saída do programa:

```
Olá, tudo bem?
7
```

174

Variáveis locais e globais

- Variáveis definidas dentro de uma função são ditas **locais**, isto é, só podem ser usadas dentro da função em que foram definidas.
- Variáveis definidas fora de função (no bloco principal) são ditas variáveis **globais**.
  - Variáveis globais podem ser acessadas dentro de uma função.
  - Para alterar uma variável global, deve-se declará-la no corpo da função com o comando **`global`**.

175

Exemplo

```
a, b = 1, 4

def imprimir():
  print(a)

def alterar1():
  a = 5

def alterar2():
  global a
  a = 5

imprimir()
1
```

```
alterar1()
print(a)
1

alterar2()
print(a)
5

# O que ocorre abaixo?
def somar():
  c = a + b

somar()
print(c)
```

176

Argumentos de funções

- Argumentos (ou parâmetros) são como variáveis que recebem seus valores iniciais do chamador.
- Essas variáveis, assim como outras definidas dentro da função são ditas **locais**, isto é, só existem no lugar onde foram definidas.
  - Ao retornar ao ponto de chamada, as variáveis locais são descartadas (desalocadas).
- Se uma função define **n** argumentos, a sua chamada deve incluir valores para todos eles.
  - Exceção: argumentos com valores **`default`**.

177

Exemplo

```
def funcao(x):
  print(x*x)

funcao(10)
100

print(x)
...
NameError: name 'x' is not defined

funcao()
...
TypeError: f() missing 1 required positional argument: 'x'
```

178

Argumentos *default*

- É possível dar valores default (predefinidos) a argumentos.
  - Se o chamador não especificar valores para esses argumentos, os valores defaults são usados.
- Formato:

```
def nome (arg_1=default1, arg_2=default2, ..., arg_n=defaultN)
```

- Se apenas alguns argumentos têm default, esses devem ser os últimos da lista.
  - Se não fosse assim, haveria ambiguidade na passagem de argumentos.

179

Exemplo

```
def f(nome, saudacao="Oi", pontuacao="!!"):
  print(saudacao+", "+nome+pontuacao)


f("Joao")
Oi, Joao!!

f("Joao","Parabens")
Parabens, Joao!!

f("Joao","Ah","...")
Ah, Joao...
```

180

Passando argumentos com nomes

- É possível passar os argumentos sem empregar a ordem de definição desde que se nomeie cada valor passado com o nome do argumento correspondente.
- Exemplo:

```
def f(nome, saudacao="Oi", pontuacao="!!"):
  print(saudacao+", "+nome+pontuacao)

f(saudacao="Valeu", nome="Joao")
Valeu,Joao!!
```

181

Alterando argumentos

- É possível alterar os parâmetros? Sim e não.
  - Os parâmetros são variáveis locais, logo pode sim ser alterados.
  - Entretanto, se um parâmetro recebe um valor que vem de uma variável global, esta não é alterada.

```
def funcao(x):              # 'x' recebe uma cópia de 'a'
  x = 5                     # e é alterada para 5.

a = 1
funcao(a)
print(a)

1                           # 'a' não é alterada.
```

182

Alterando argumentos

- Quando passamos uma variável do tipo **lista** (list) como parâmetro, estamos passando uma *referência* para um valor do tipo lista.
  - Nesse caso, alterar o parâmetro pode refletir em alterar o "*valor*" da variável global.
  - Na verdade, o "*valor*" da variável do tipo lista é uma referência que *não muda*.
  - Este caso é idêntico a termos duas variáveis *se referindo* ao mesmo valor na memória.

183

Exemplo

```
def funcao(x):
  x[:] = [5]

a = [1, 2, 3, 4]
print(a)
Saída: [1, 2, 3, 4]

funcao(a)
print(a)
Saída: [5]

b = a
b[:] = [7]
print(a)
Saída: [7]
```

184

Lista de parâmetros variáveis (1)

- Como sabemos, a quantidade de parâmetros enviados na chamada deve ser a mesma quantidade de parâmetros recebidos na função.
- Como podemos enviar quantidade variada de valores?
- Usamos como último parâmetro da função uma variável precedido de asterisco * .
  - Neste caso, todos os valores passados a partir daquele são postos em uma tupla.

185

Lista de parâmetros variáveis (2)

- Exemplo:

```
def imprime(nome, *atributos):
  print(nome, atributos)

imprime ("ana", 9.0, 8.0, 7.0)
Saída: ana (9.0, 8.0, 7.0)
```

186

Lista de parâmetros variáveis (3)

- Exemplo:

```
def mostra_media(*valores):
  soma = 0.0
  for x in valores:
    soma +=x
  print(f"Média: {soma / len(valores)}")

mostra_media(9.0, 8.0, 7.0)
Saída: Média: 8.0
```

187

Lista de parâmetros variáveis (4)

- Com o último parâmetro da função precedido de dois asteriscos **, todos os valores passados como *key=valor* a partir daquele são postos em um dicionário.

```
def funcao(a, b, **c):
  print(a, b, c)

funcao(1, 2, 3)
TypeError: funcao() takes 2 positional arguments but 3 were given

funcao(1, 2, x=3)
1 2 {'x': 3}

funcao(1, 2, x=3, y=5)
1 2 {'x': 3, 'y': 5}
```

188

Lista de parâmetros variáveis (5)

- Pode passar valores de uma tupla para preencher parâmetros posicionais de uma função, bastando para isso precedê-la de *.
- Um dicionário pode ser usado para preencher parâmetros por chaves bastando para isso precedê-lo de **.
- É preciso tomar cuidado para não abusar!

```
def funcao(a, b, *c, **d):
    print(a, b, c, d)
```

189

Lista de parâmetros variáveis (5)

```
def funcao(a, b, *c, **d):
  print(a, b, c, d)

funcao(*[1, 2, 3, 4, 5])
Saída: 1 2 (3, 4, 5) {}

funcao(**{"a":1, "b":2, "c":3, "d":4})
Saída: 1 2 () {'c':3, 'd':4}

funcao(1, 2, 3, **{"d":1})
Saída: 1 2 (3,) {'d':1}

funcao(1, 2, 3, **{"a":1})
...
TypeError: funcao() got multiple values for argument 'a'
```

190

Retorno de função

- Uma função computa um ou mais valores e retorna ou não um ou mais valores como resultado. Para retornar, usa-se o comando **`return`**.

  `return` `<expressão>`

  - `<expressão>` (opcional) designa o(s) valores a ser(em) retornado(s)

- Achando **`return`**, a função termina de imediato e o controle do programa volta ao ponto onde a função foi chamada.
- Se uma função termina sem nenhum valor de retorno ter sido especificado, o valor de retorno é **`None`**.

191

Retorno de função

```
def soma(a, b):
  return a + b

dobro(x):
  return 2 * x


soma(1, 2)
Saída: 3

dobro(10)
Saída: 20
```

192

Retorno múltiplos valores

- Apesar de uma função executar apenas um retorno (return) pode-se retornar mais de um valor.
- Como exemplo, o que imprime o código a seguir?

```
def calculadora1(a, b):
  return a+b, a-b

def calculadora2(a, b):
  return {"soma": a+b, "subtracao": a-b}

print(calculadora1(5, 2))
print(calculadora2(5, 2))
```

193

Escopo

- Escopo é o nome que se dá ao conjunto de nomes acessíveis de um determinado ponto de um programa.
  - Também é chamado de espaço de nomes ou namespace.
- Um programa começa em um escopo (chamado escopo global) enquanto que cada função acrescenta um escopo próprio (local).
  - Módulos e classes também definem escopos.
- Ao se fazer acesso a um nome, todos os escopos, do mais interno para o mais externo, são consultados.
  - Isto explica por que definir uma variável numa função pode fazer com que uma variável global deixe de ser acessível.

194

Funções definidas em função

- Funções podem ser definidas dentro de funções.
- Se uma função produto() é definida dentro de uma função funcao(), ela tem acesso ao seu próprio escopo (em primeiro lugar) e também ao escopo de funcao().
- Exemplo:

```
def funcao(x):
  def produto(y):
    return x*y
  return produto(2)


print(f"Valor: {funcao(4)}")
Saída: Valor: 8
```

195

Funções definidas em função (2)

- Se uma função **g()** foi definida dentro de outra função **f()**, então, se **g()** é armazenada em uma variável ou transmitida para outra função, então **g()** carrega consigo os valores do escopo de **f()**, mas não o escopo global. Exemplo:

```
def f(y):
  def g(z):
    return x*y*z
  return g

x = 2
h = f(3)
print h(1)                  # imprime 6

x = 3
print h(1)                  # imprime 9
```

196

Função *vars()*

- O dicionário obtido com a função **vars()** pode ser usado para ter acesso a todas as variáveis definidas num escopo. Exemplo:

```
vars()
{'__builtins__': <module '__builtin__' (built-in)>,
'__name__': '__main__', '__doc__': None}

def f():
  x = 1
  print vars()

vars()
{'f': <function f at 0xb6e7f56c>, '__builtins__':
<module '__builtin__' (built-in)>, '__name__':
'__main__', '__doc__': None}

f()
Saída: {'x': 1}
```

197

Documentando funções

- Ao invés de usar comentários para descrever o que uma função faz, é mais vantajoso usar ***docstrings***.
- Uma constante **string** escrita logo após o cabeçalho (def) da função.
- Permite o acesso à documentação a partir do interpretador, usando a notação função **.__doc__**

198

Documentando funções

```
def fat(n):
  "Retorna o fatorial de n.“
  for i in range(n-1,1,-1):
    n*=i
  return n

fat(4)
Saída: 24

print(fat.__doc__)
Saída: Retorna o fatorial de n.
```

199